

Prompt Iteration Log: Next.js Form Engineering

Target Task (FL-01 Audit): Generating a fully validated, accessible, and conflict-free user profile settings component within a Next.js App Router environment.

Naive Version (Baseline)

Prompt:

"Write a form for user settings in React."

Output Excerpt:

```
```jsx
const SettingsForm = () => {
 const [name, setName] = useState("");
 return (<form><input type="text" value={name} onChange={e => setName(e.target.value)}
/><button>Submit</button></form>)
}
```
```

What Changed & Why:

Baseline starting point representing zero context or architectural specifications.

Observed Output Difference:

Generated highly generic React 16 style code utilizing unmanaged standard state parameters. Failed to provide styling hooks, semantic accessibility landmarks, integration patterns, or routing syntax.

Iteration 1: Role Assignment

Prompt:

"Act as an expert frontend engineer specializing in Next.js App Router architectures. Write a client-side settings form component that handles user name and phone updates."

Output Excerpt:

```
```tsx
// app/settings/page.tsx
"use client";
import { useState } from "react";
export default function SettingsForm() {
 const [phone, setPhone] = useState("");
 // ... standard inputs
}
```
```

What Changed & Why:

Assigned a specific framework expert profile to push the AI past basic vanilla React generation.

Observed Output Difference:

The output immediately updated its framework logic: it discarded obsolete boilerplate and generated structural Next.js App Router file routing headers along with the proper mandatory 'use client' boundary string.

Iteration 2: Context & Motivation

Prompt:

"Act as an expert frontend engineer specializing in Next.js App Router architectures. Write a client-side settings form component that handles user name and phone updates. Context: This form must integrate cleanly with our existing backend schema without causing dependency or merge blocks in package.json. You must use react-hook-form paired with zod for schema validation."

Output Excerpt:

```
````tsx
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import * as z from "zod";
const schema = z.object({ phone: z.string() });
````
```

What Changed & Why:

Introduced dependency parameters and real-world system constraints to prevent arbitrary code choices that disrupt local package installations.

Observed Output Difference:

The component shifted entirely to modern hook schema wrappers, completely abandoning native unstructured text states. However, the data verification patterns remained completely blank.

Iteration 3: Few-Shot Examples

Prompt:

"Act as an expert frontend engineer... Context: Use react-hook-form and zod validation. Example of what a highly robust international validation rule looks like in Zod: phone: z.string().regex(/^\+[1-9]\d{1,14}\$/, "Invalid E.164 phone number"). Only accept valid structures matching this precise format requirement."

Output Excerpt:

```
````tsx
const schema = z.object({
 name: z.string().min(2),
 phone: z.string().regex(/^\+[1-9]\d{1,14}$/, "Invalid E.164 phone number")
})
```

```
});
""
```

**What Changed & Why:**

Provided an exact code reference sample to define data boundary constraints.

**Observed Output Difference:**

The regex rule pattern successfully evolved to evaluate against rigorous E.164 parameters. But the AI hyper-focused on code generation, skipping UI implementation details like accessibility properties and error alerts.

## Iteration 4: Output Structure

**Prompt:**

*"Act as an expert frontend engineer... Context: Use react-hook-form and zod. Enforce E.164 phone validation rules. Output Structure Requirement: The generated layout must strictly alternate between a structural semantic HTML section (<form>, <label>, <input>) and inline UI visibility error triggers matching this explicit pattern: {errors.fieldName && <p className=\"text-red-500 text-sm\">{errors.fieldName.message}</p>}."*

**Output Excerpt:**

```
````tsx  
// CRITICAL ARCHITECTURAL CONFLICT ENCOUNTERED  
// The code structure generated clean UI layouts but bundled an incompatible Next.js form state  
hook:  
export default function Form() {  
  const [state, action] = useFormState(saveAction, null);  
  ""
```

What Changed & Why:

Specified precise template parameters to mandate visual error message rendering.

Observed Output Difference:

HONEST FAILURE STEP: While error layouts rendered correctly with Tailwind styling, the AI hallucinated structural parameters and introduced an experimental Hook that broke layout tree rules. It made things worse.

Iteration 5: Step Decomposition

Prompt:

"Act as an expert frontend engineer... Context: Use react-hook-form and zod. Step Decomposition Rules: 1. Evaluate package dependencies and declare pure schema layers. 2. Build local state execution paths ensuring NO conflicting React experimental server hooks are introduced. 3. Map validation error layouts under each field. 4. Verify accessibility setups by attaching linked IDs, htmlFor labels, and functional aria-invalid state triggers to every element."

Output Excerpt:

```
````tsx
"use client";
import { useForm } from "react-hook-form";
// Pure standard submit actions mapped with complete HTML structure and deep aria-invalid state
triggers
return (
 <div>
 <label htmlFor="phone">Phone Number</label>
 <input id="phone" aria-invalid={errors.phone ? "true" : "false"} ... />
 </div>
)
```

**What Changed & Why:**

Forced the model to review architectural layers sequentially, prohibiting experimental bindings while requiring accessibility compliance.

**Observed Output  
Difference:**

The output completely eliminated conflicting code patterns, defaulting back to reliable client actions. It successfully added fully accessible attributes, proper labeling scopes, and robust structural safety tags.

## Cross-Model Comparison (Claude vs. ChatGPT)

| Metric/Dimension | Claude 3.5 Sonnet Output                                                                                                 | ChatGPT (GPT-4o) Output                                                                                             |
|------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Tone & Style     | Highly technical, direct, plain, code-first with zero introductory conversational filler.                                | Slightly conversational, includes boilerplate introductory summaries and conversational concluding sign-offs.       |
| Accuracy         | Strictly honored explicit constraints; generated flawless, isolated E.164 regex logic without breaking scope boundaries. | Initially tried to drop the specific E.164 regular expression requirement, defaulting to a looser validation check. |
| Structure        | Perfect structural isolation between structural schema definitions and pure, accessible semantic JSX markups.            | Bundled extra helper abstractions inside the main component block, increasing visual complexity.                    |
| Failure Points   | Missed a visual loading latency placeholder state element, though the component compiled cleanly.                        | Hallucinated an extra import block pointing to a non-existent utility folder.                                       |

# Final Reusable Engineered Template

Act as an expert frontend engineer specializing in Next.js App Router architectures.

Task: Generate a clean, standalone production-ready client-side form component for updating [INSERT\_FORM\_PURPOSE] profile parameters (specifically targeting: [INSERT\_INPUT\_FIELDS]).

Architectural Requirements:

1. Must be a Next.js App Router client component ("use client").
2. Form state management must be powered exclusively by 'react-hook-form'.
3. Input validations must be processed via a Zod verification schema passed through a 'zodResolver'.

Constraint Rules:

- [FIELD\_1]: [SPECIFY\_CONSTRAINTS, e.g., required, character length]
- [FIELD\_2]: [SPECIFY\_CONSTRAINTS, e.g., international E.164 phone regex verification format]

UI & Accessibility Standards:

- Must use clean, semantic HTML layout elements (<form>, <label>, <input>).
- All input elements must feature corresponding 'htmlFor' label connections and dynamic 'aria-invalid' bindings linked to active error fields.
- Validation errors must render immediately below their respective inputs using this specific styling structure:  
{errors.fieldName && <p className="text-red-500 text-sm mt-1">{errors.fieldName.message}</p>}

Verification Checks:

Review your code before outputting. Ensure no older React experimental hooks or conflicting state managers are bundled. Do not combine this with raw server action bindings inside the form element—process everything securely through a standard, isolated onSubmit handler function.